accenture

Code.Hub

Project Assignment

GTGH | AI and Data Engineering

AI and Data Capstone Project

Enterprise Data Platform and Analytics Solution

Common technical specification for five teams

4 teams • 5 participants per team • Equal technical difficulty

Contents

1. Outline 3

2. Business Problem 3

   2.1 Fragmented Data Pipeline 3

   2.2 Poor Data Quality 3

   2.3 Limited Analytical Capability 3

3. Objective 4

4. Dataset 4

5. Target Architecture 5

6. Scope of Work 5

   6.1 Databricks: Bronze Ingestion 5

   6.2 Databricks: Silver Preparation 5

   6.3 Databricks: Gold Dimensional Model (Delta) 6

   6.4 Microsoft Fabric: Semantic Model and Report 6

   6.5 LangChain Core: Talking to the LLM and Tools 6

   6.6 Azure Fundamentals 6

   6.7 REST APIs, FastAPI and Azure Integration 7

   6.8 Applied DevOps: CI/CD 7

7. Deliverables 8

8. Project Management 8

9. Evaluation Criteria 9

accenture

Code.Hub

1. Outline

Each team will design, implement and demonstrate an end-to-end enterprise data and analytics solution. The shared technical path is source ingestion into Azure Databricks, progressive refinement through Bronze, Silver and Gold Delta layers, and delivery of a governed Microsoft Fabric semantic model and Power BI report. The solution will also expose selected analytical capabilities through secured FastAPI endpoints and a LangChain Core interface in which the LLM can call approved data tools.

All teams follow the same architecture, scope, deliverables, project-management controls and evaluation rubric. The business domain, source data, analytical questions and measures differ by team, keeping the technical workload comparable while producing five distinct solutions.

Stage	Expected outcome
Discover	Understand the business problem, users, decisions, data sources and success measures.
Engineer	Build repeatable Databricks pipelines from raw files to trusted Delta tables.
Model	Create a dimensional Gold model with documented facts, dimensions and measures.
Analyze	Build a Fabric semantic model and an interactive Power BI report.
Validate	Test data quality, reconciliation, security, performance and business usability.
Demonstrate	Present the architecture, lineage, insights and a working end-to-end refresh.

2. Business Problem

Organizations frequently possess enough operational data to make better decisions, but the data is distributed across files and systems, defined inconsistently and consumed through manually prepared reports. The capstone addresses this gap by establishing a reliable analytical path from heterogeneous source data to governed business insight.

2.1 Fragmented Data Pipeline

CSV and JSON extracts are delivered independently, with no consistent ingestion contract, processing schedule or lineage. Analysts repeat manual preparation, source changes can break downstream work, and it is difficult to reproduce how a reported value was produced.

- Multiple source formats and inconsistent delivery conventions.
- Manual or duplicated transformation logic.
- Limited observability of refresh status, failures and record movement.
- No clear separation between raw, validated and business-ready data.

2.2 Poor Data Quality

Source records may contain missing identifiers, duplicates, invalid dates, inconsistent units, malformed categories and late-arriving events. Without explicit controls, these defects propagate into metrics and undermine trust in the report.

- No systematic schema, completeness, uniqueness or referential-integrity checks.
- Business rules are implicit and difficult to audit.
- Rejected or quarantined records are not traceable to a resolution process.
- Source-to-target totals are not routinely reconciled.

2.3 Limited Analytical Capability

Operational files can answer isolated questions, but they do not provide a reusable model for trends, comparisons, drill-down analysis or shared KPI definitions. Decision-makers need a semantic layer that presents consistent measures, meaningful dimensions and report navigation suited to the business problem.

3. Objective

The objective is to deliver a production-minded analytical solution that converts heterogeneous source data into trusted, explainable and decision-ready information. The solution must demonstrate engineering quality as well as business value. It must integrate data engineering, analytics, APIs, tool-enabled LLM interaction, Azure security fundamentals and automated delivery.

- Automate incremental ingestion and retain the original source representation in Bronze.
- Standardize, validate, deduplicate and conform records in Silver.
- Publish a documented dimensional model in Gold using Delta tables.
- Create a Microsoft Fabric semantic model with reusable DAX measures and clear relationships.
- Deliver a Power BI report that answers the assigned business questions and supports drill-down.
- Demonstrate lineage, security, testing, monitoring and a repeatable deployment process.
- Expose selected business queries through validated REST endpoints built with FastAPI and Pydantic.
- Use LangChain Core messages, prompts and tool calling to let the LLM invoke only approved analytical tools.
- Apply Azure identity, resource organization and secret-management practices appropriate to the solution.
- Automate linting, tests, build and staged deployment through GitHub Actions or Azure DevOps Pipelines.

4. Dataset

Each team receives or generates a small but realistic dataset for its assigned domain. The data must be large enough to reveal quality and modeling issues but compact enough to refresh during a live demonstration. Synthetic values must remain internally consistent and contain no real personal or confidential information.

Data class	Minimum expectation	Examples
Transactional CSV	At least two related files with dates, identifiers, quantities or monetary values	Incidents, loans, work orders, purchase orders, complaints
Reference CSV	At least one master-data file used for enrichment	Assets, customers, equipment, suppliers, services
Semi-structured JSON	At least one event stream or nested dataset	Monitoring, transactions, sensors, shipment events, service usage
Metadata	Source filename, ingestion timestamp, batch identifier and source-system key	Audit and lineage columns

Minimum data-design requirements:

A data dictionary describing fields, types, keys, business meaning and sensitivity.
A source-to-target mapping for all Silver and Gold tables.
Deliberately injected quality defects to demonstrate detection and remediation.
A documented refresh strategy, including the treatment of late-arriving and corrected records.
At least 10,000 transactional or event records in total.

5. Target Architecture

The architecture separates ingestion, data preparation, business modeling and consumption. Delta Lake provides reliable storage and table history in Databricks; Microsoft Fabric provides the governed semantic and reporting layer.

Layer	Purpose
Sources	CSV and JSON files, reference/master data and optional document sources
Databricks Bronze	Raw, append-only Delta tables with ingestion metadata and audit columns
Databricks Silver	Validated, standardized, deduplicated and conformed Delta tables
Databricks Gold	Star schema: fact tables, dimensions, business measures and aggregates
Microsoft Fabric	OneLake/Lakehouse or Warehouse access and a governed semantic model
Power BI	Interactive report with executive summary, analysis pages and drill-through
FastAPI integration	Validated REST endpoints for selected analytical queries and service health
LangChain Core	Messages, prompts, structured output and approved tool calling over the analytical services
Azure platform	Resource Groups, IAM, Storage, Compute, Networking and Key Vault
CI/CD	Automated lint, test, build and controlled deployment stages

Required cross-cutting capabilities include environment-based configuration, secrets outside source control, role-based access, source-to-report lineage, job monitoring, automated tests, version control and reproducible deployment.

6. Scope of Work

The following scope is mandatory for every team. Optional enhancements may be added only after the complete baseline solution works end to end.

6.1 Databricks: Bronze Ingestion

- Create parameterized ingestion notebooks or Python modules for CSV and JSON sources.
- Persist source data as Bronze Delta tables without applying business-level transformations.
- Add ingestion timestamp, batch ID, source filename, record hash and load status.
- Implement schema validation and a controlled response to schema drift.
- Support idempotent reruns and either incremental loading or a clearly justified full-refresh pattern.
- Log input counts, written counts, rejected counts, duration and failure details.

Bronze acceptance condition: the team can trace any downstream record to its source file and ingestion batch and can rerun a failed batch without creating uncontrolled duplicates.

6.2 Databricks: Silver Preparation

- Standardize names, types, dates, time zones, codes, currencies and units.
- Apply domain rules, deduplicate records and resolve invalid or missing keys.
- Flatten or normalize relevant JSON structures and join reference data.
- Implement data-quality expectations for completeness, validity, uniqueness and referential integrity.
- Route invalid records to a quarantine table with reason, rule and processing timestamp.
- Document the treatment of slowly changing reference attributes and late-arriving events.

Silver acceptance condition: conformed records are analytically consistent, quality results are measurable, and rejected records remain visible and actionable.

6.3 Databricks: Gold Dimensional Model (Delta)

Create a star schema in Delta. The model must state the grain of every fact table and use conformed dimensions wherever the domain supports them.

- At least one central fact table and four meaningful dimensions, including a date dimension.
- Surrogate keys where appropriate and documented handling of unknown dimension members.
- Business-ready measures, status logic and aggregates aligned with the assigned questions.
- Optimized Delta layout appropriate to the dataset, supported by a brief performance rationale.
- Reconciliation from Silver to Gold for row counts and critical totals.
- Table and column descriptions sufficient for another analyst to use the model.

Gold acceptance condition: business measures can be reproduced from documented facts and dimensions without embedding inconsistent logic in individual report visuals.

6.4 Microsoft Fabric: Semantic Model and Report

Connect Microsoft Fabric to the curated Gold model using the access pattern agreed with the instructor. Build a semantic model that hides technical complexity and exposes governed business terminology.

- Define relationships, hierarchies, formats, descriptions and a dedicated measures table.
- Create at least eight explicit DAX measures, including volume, rate or ratio, trend and exception measures.
- Use a proper date table and validate filter direction and measure behavior.
- Implement row-level security when the business case contains region, branch, team or customer ownership boundaries.
- Create at least three report pages: executive overview, analytical detail and exceptions/drill-through.
- Include filters, tooltips, accessible titles, clear units and a visible last-refresh indicator.
- Validate report totals against Gold tables and document performance observations.

6.5 LangChain Core: Talking to the LLM and Tools

Build a small, auditable LLM integration using langchain-core. The LLM is an interface to approved analytical capabilities; it must not bypass the semantic model, fabricate data or perform consequential business actions.

- Use system, human, AI and tool messages correctly when constructing the conversation.
- Create prompt templates that define the business context, available evidence, response format and limitations.
- Define at least two typed tools and bind them to the chat model. One tool must retrieve a KPI or aggregate and one must retrieve filtered detail or metadata.
- Detect tool calls, validate their arguments, execute the approved function and return a ToolMessage to the model.
- Return structured output that distinguishes answer, evidence, filters, source or measure, and limitations.
- Handle malformed arguments, unavailable data, tool errors and requests outside the permitted scope.

Acceptance condition: a user question can trigger a validated tool call, receive a result grounded in the Gold or semantic layer and produce an evidence-based response with a traceable execution path.

6.6 Azure Fundamentals

- Document and apply the Azure foundation used to host and secure the integration components.
- Identify the Azure account, subscription and Resource Group boundary for the solution.
- Define role-based access using least privilege and explain the responsibilities of users, developers and the runtime identity.

- Select and justify the required Storage, Compute and Networking services.
- Store application secrets and credentials outside source control, using Azure Key Vault for the deployed solution.
- Provide an environment diagram and a concise resource inventory with naming and configuration responsibilities.

6.7 REST APIs, FastAPI and Azure Integration

Expose a documented integration layer that connects consumers to approved analytical and LLM-assisted capabilities.

- Apply REST fundamentals: resources, HTTP verbs, status codes and consistent error responses.
- Implement FastAPI routes with Pydantic request and response models, validation and exception handling.
- Provide OpenAPI/Swagger documentation and a Postman collection or equivalent executable API examples.
- When consuming external APIs, implement timeouts, pagination where relevant, bounded retries and rate-limit handling.
- Protect endpoints with an agreed mechanism such as API keys during development and OAuth2 bearer tokens or JWT validation for the deployed design.
- Include health and readiness endpoints, correlation IDs and structured logs without exposing secrets or sensitive data.
- Connect the API end to end with the Gold/semantic layer and the LangChain tools; mock unsupported cloud dependencies only when clearly documented.

Minimum API: GET /health, POST /analytics/query, POST /assistant/query and one domain-specific endpoint. Quiz 1 is used as the knowledge checkpoint for LangChain Core, Azure fundamentals, REST/API security and CI/CD foundations.

6.8 Applied DevOps: CI/CD

Implement the delivery workflow with GitHub Actions or Azure DevOps Pipelines. The pipeline definition must be versioned with the project.

- Configure workflow triggers for pull requests and the main branch.
- Define separate lint, test, build and deployment jobs with clear dependencies and failure behavior.
- Build the API application and, where used, its container image as a reproducible artifact.
- Use repository or pipeline secret stores; never commit tokens, passwords, connection strings or model credentials.
- Use environments, approvals and staged deployment for development and the target release environment.
- Document rollback, configuration promotion and the evidence required before approval.

Acceptance condition: a controlled source change triggers the pipeline, quality gates execute automatically, an artifact is created and the deployment stage is either completed or held for the configured approval.

7. Deliverables

Deliverable	Required evidence
Business and data brief	Problem statement, users, decisions, KPIs, dataset inventory and data dictionary
Architecture and lineage	Source-to-report-to-API diagram covering Databricks, Fabric, FastAPI, LangChain and Azure resources
Databricks implementation	Versioned notebooks/modules, job configuration and Bronze/Silver/Gold Delta tables
Quality evidence	Rules, test results, quarantine examples and reconciliation summary
Dimensional model	Grain statements, star-schema diagram, keys, dimensions, facts and mapping
Fabric and Power BI	Semantic relationships, DAX measures, security and at least three report pages
LangChain Core integration	Prompts, structured output, at least two bound tools, tool-result handling and grounded-response examples
FastAPI integration	Validated endpoints, OpenAPI docs, error handling, API security and executable request examples
Azure design	Subscription/Resource Group boundary, IAM, service choices, Key Vault and environment diagram
CI/CD pipeline	Versioned YAML with triggers, lint, tests, build, secrets, environments, approvals and deployment stage
Testing and operations	Unit/integration tests, monitoring/log evidence, refresh/run procedure and known limitations
Final presentation	15-minute demonstration plus 5-minute questions; every participant contributes

8. Project Management

Teams will work in short iterations with visible ownership and objective acceptance criteria. All participants must understand the complete solution even when they lead different workstreams.

Role	Primary ownership
Participant 1 - Data engineering	Source contracts, Bronze/Silver pipelines, audit metadata, quality and Databricks jobs
Participant 2 - Analytics	Gold model, Fabric semantic model, DAX, Power BI and reconciliation
Participant 3 - AI and API	LangChain Core prompts/tools, structured output, FastAPI, Pydantic and API security
Participant 4 - Cloud and DevOps	Azure architecture, IAM/Key Vault, CI/CD, deployment, monitoring and integration tests
Participant 5 - Project manager	Project plan and milestones, task coordination, dependency and risk tracking, stakeholder communication, acceptance criteria, and final delivery

Use a shared backlog with owner, priority, acceptance criteria and status.
Use pull requests or equivalent peer review for material changes.
Record architectural decisions and scope changes in the repository.
Maintain a risk log covering data availability, access, quality, integration and schedule.

Demonstrate a working increment at every checkpoint; slides alone do not establish completion.

9. Evaluation Criteria

A technically incomplete pipeline cannot receive a high overall score solely because the report is visually polished. Likewise, a correct pipeline must still demonstrate understandable business measures and a usable analytical outcome.

Criterion	Weight	What is evaluated
Business framing and KPI design	8%	Clear decisions, measurable value and relevant analytical questions
Bronze ingestion	12%	Repeatability, metadata, schema handling, idempotency and observability
Silver preparation and quality	15%	Correct transformations, rules, quarantine, tests and traceability
Gold dimensional model	15%	Grain, star-schema quality, keys, measures, reconciliation and performance
Fabric semantic model and report	15%	Relationships, DAX, insight, usability, security and validation
LangChain Core and tools	10%	Prompts, tool binding/calling, structured output, evidence and failure handling
FastAPI, Azure and security	10%	REST design, validation, authentication, IAM, Key Vault and integration
CI/CD and operations	8%	Triggers, lint/test/build jobs, secrets, environments, approvals and deployment
Demonstration and teamwork	7%	End-to-end evidence, shared understanding and response to questions

Bonus Challenge' (5%) in the project

Build a simple Knowledge Graph from the domain of your team and expose one graph-based query through the AI assistant.

Team Business Cases

The following assignments apply the same common specification to five domains of equivalent difficulty. The team must implement the same Bronze-Silver-Gold pipeline, Fabric semantic model, secured FastAPI layer, LangChain Core tools, Azure design, governance controls, tests and CI/CD process described in the common specification.

Team 1 - Intelligent IT Incident Analytics

A large organization needs reliable analysis of incident volume, severity, affected services, resolution performance and recurring operational causes.

Design element	Team-specific implementation
Source data	incidents.csv, assets.csv, teams.csv and monitoring_events.json
Primary analytical grain	One incident event or incident status transition at a defined timestamp
Gold fact tables	fact_incident, fact_monitoring_event
Gold dimensions	dim_date, dim_asset, dim_service, dim_team, dim_severity, dim_resolution_category
Required KPIs	Incident count; open backlog; P1/P2 rate; mean time to acknowledge; mean time to resolve; SLA compliance; recurrence rate; affected users
Report pages	Executive operations; incident trends and SLA; service/asset drill-through; recurring causes and exception queue
Priority data-quality rules	Duplicate incident IDs; invalid timestamps; closed incidents without resolution; unknown asset keys; negative durations

Team 2 - Banking Loan Portfolio Analytics

A bank needs a governed view of applications, approvals, risk bands, customer affordability and portfolio performance without exposing unnecessary customer detail.

Design element	Team-specific implementation
Source data	customers.csv, loan_applications.csv, repayments.csv and transactions.json
Primary analytical grain	One loan application and, separately, one scheduled or actual repayment transaction
Gold fact tables	fact_loan_application, fact_repayment
Gold dimensions	dim_date, dim_customer_segment, dim_product, dim_branch, dim_risk_band, dim_application_status
Required KPIs	Applications; requested amount; approval rate; average decision time; debt-to-income band; delinquency rate; outstanding balance; manual-review rate
Report pages	Portfolio overview; application funnel; risk and affordability; branch/product drill-through; exceptions
Priority data-quality rules	Missing consent/status; inconsistent amounts; impossible dates; duplicate applications; orphan repayments; out-of-range risk scores

Team 3 - Predictive Maintenance Analytics

An industrial company needs to understand equipment health, abnormal sensor behavior, maintenance effectiveness, downtime and recurring failure modes.

Design element	Team-specific implementation
Source data	equipment.csv, maintenance_history.csv, work_orders.csv and sensor_events.json
Primary analytical grain	One sensor reading at a timestamp and one maintenance work order
Gold fact tables	fact_sensor_reading, fact_maintenance_work_order, fact_downtime
Gold dimensions	dim_date, dim_equipment, dim_site, dim_equipment_type, dim_failure_mode, dim_maintenance_type
Required KPIs	Asset availability; unplanned downtime; mean time between failures; mean time to repair; overdue work orders; alarm rate; maintenance cost; repeat-failure rate
Report pages	Fleet health; sensor trends; maintenance performance; equipment drill-through; critical exceptions
Priority data-quality rules	Out-of-range readings; unit inconsistency; missing equipment keys; overlapping downtime; closure before creation; duplicate sensor events

Team 4 - Supply Chain Disruption Analytics

A company needs visibility into delayed shipments, affected purchase orders, supplier reliability, inventory exposure and mitigation priorities.

Design element	Team-specific implementation
Source data	suppliers.csv, purchase_orders.csv, products.csv and shipment_events.json
Primary analytical grain	One purchase-order line and one shipment status event
Gold fact tables	fact_purchase_order_line, fact_shipment_event, fact_inventory_exposure
Gold dimensions	dim_date, dim_supplier, dim_product, dim_location, dim_shipment_status, dim_risk_level
Required KPIs	Order value; on-time delivery rate; average delay; supplier fill rate; exposed value; delayed quantity; critical-item count; alternate-supplier coverage
Report pages	Supply overview; delay and exposure; supplier scorecard; product/location drill-through; mitigation queue
Priority data-quality rules	Unknown supplier/product; negative quantities; invalid status sequence; delivery before order; currency inconsistency; duplicate order lines